

Video 2 — Hướng dẫn cài đặt môi trường

Phiên bản: 1.0

Ngày cập nhật: 28/07/2026

Thời lượng mục tiêu: 14–17 phút

Đối tượng: Sinh viên thực hành API Testing và Contract Testing trên Windows, macOS hoặc Linux

Mục lục

- 1. Mục tiêu và kết quả đầu ra
- 2. Chuẩn bị trước khi quay
- 3. Tổng quan timeline
- 4. Kịch bản chi tiết
- 5. Bảng kiểm kết thúc video
- 6. Xử lý lỗi thường gặp
- 7. Tài liệu tham khảo

1. Mục tiêu và kết quả đầu ra

Sau video này, người xem có thể:

- Cài Node.js bản LTS và xác nhận `node`, `npm` hoạt động.
- Cài và mở Postman Desktop.
- Cài Visual Studio Code và extension **REST Client** đúng tác giả **Huachao Mao**.
- Cài hoặc kiểm tra Git, clone đúng repository của nhóm và mở repo bằng VS Code.
- Tạo workspace Contract Testing trên PactFlow, lấy Broker URL và API token.
- Tạo file `src/.env` từ file mẫu, cấu hình thông tin PactFlow và kiểm tra kết nối mà không làm lộ token.

Phạm vi video là **chuẩn bị môi trường**. Việc chạy API, gửi request Postman, publish Pact và provider verification được trình bày ở các video thực hành tiếp theo.

Quy ước dùng trong video

- Luồng thao tác chính được quay trên **Windows 10/11, PowerShell**.
- Các lệnh tương đương cho macOS/Linux được hiển thị bằng callout hoặc phụ đề.
- Giá trị có dạng `<...>` là placeholder; người xem phải thay bằng giá trị của mình.
- Không quay, đọc thành tiếng hoặc dán API token thật lên màn hình.

2. Chuẩn bị trước khi quay

2.1. Tài nguyên cần mở sẵn

- Trang tải Node.js: <https://nodejs.org/en/download>
- Trang tải Postman: <https://www.postman.com/downloads/>
- Trang tải VS Code: <https://code.visualstudio.com/download>
- Trang tải Git: <https://git-scm.com/downloads>
- Marketplace REST Client: <https://marketplace.visualstudio.com/items?itemName=humao.rest-client>
- PactFlow: <https://pactflow.io/>
- Repository: https://github.com/se-bros/Software_Testing_api_contract_testing

2.2. Chuẩn bị tài khoản và dữ liệu quay

- Có email dùng để đăng ký hoặc đăng nhập SmartBear/PactFlow.
- Đăng xuất PactFlow trước khi quay nếu muốn minh họa toàn bộ luồng đăng ký.
- Tắt thông báo hệ điều hành, email và ứng dụng chat.
- Dùng workspace thử nghiệm; không hiển thị dữ liệu hoặc token của workspace thật.
- Chuẩn bị sẵn một token giả để dán khi minh họa, ví dụ `pactflow_token_HIDDEN`.
- Đóng terminal đang chứa biến môi trường hoặc command history có secret.

2.3. Quy tắc bảo mật bắt buộc khi quay

1. Khi đến trang **API Tokens**, đừng quay hoặc phủ blur vùng token trước khi bấm **Copy Token Value**.
2. Không chạy lệnh kiểu `echo $PACT_BROKER_TOKEN` hay `Write-Output $env:PACT_BROKER_TOKEN`.
3. Chỉ hiển thị token dưới dạng `<your-read-write-token>` hoặc `.....`.
4. File thật là `src/.env`; file này đã được `.gitignore` loại trừ. Chỉ commit `src/.env.example`.
5. Nếu token vô tình xuất hiện trong bản quay, phải thu hồi/regenerate token trước khi phát hành video.

3. Tổng quan timeline

Thời gian	Phân đoạn	Kết quả trên màn hình
00:00–00:40	Mở đầu	Nêu mục tiêu và bộ công cụ cần cài
00:40–03:30	Node.js	<code>node --version</code> và <code>npm --version</code> chạy thành công
03:30–05:40	Postman	Postman Desktop mở được
05:40–07:40	VS Code + REST Client	Extension đúng publisher ở trạng thái Installed
07:40–10:10	Git và clone repo	Repo được clone, mở trong VS Code
10:10–13:50	PactFlow Broker	Workspace, Broker URL và token được tạo
13:50–16:20	Cấu hình và kiểm tra	<code>src/.env</code> hợp lệ, Broker trả HTTP thành công
16:20–17:00	Tổng kết	Hoàn thành checklist môi trường

Nếu máy đã có một công cụ, vẫn chạy bước kiểm tra phiên bản rồi chuyển tiếp; không cần gỡ và cài lại.

4. Kịch bản chi tiết

Cảnh 1 — Mở đầu và giới thiệu kết quả (00:00–00:40)

Hình ảnh trên màn hình

- Hiện title card: “Video 2 — Hướng dẫn cài đặt môi trường”.
- Chuyển sang slide có sáu mục: Node.js, Postman, VS Code, REST Client, Git/GitHub, PactFlow.

Lời thoại

Chào các bạn. Trong video này, chúng ta sẽ chuẩn bị đầy đủ môi trường để thực hành API Testing và Contract Testing với repository của nhóm SEBros. Mình sẽ lần lượt cài Node.js, Postman, VS Code REST Client, clone source code bằng Git, sau đó tạo và cấu hình tài khoản PactFlow Broker. Cuối video, chúng ta sẽ kiểm tra từng thành phần để chắc chắn môi trường đã sẵn sàng.

Chú thích dựng phim

- Hiện URL repository ở cuối màn hình.
- Thêm callout: “Không chia sẻ API token trong video hoặc source code.”

Cảnh 2 — Cài Node.js LTS (00:40–03:30)

Bước 2.1 — Kiểm tra Node.js đã có hay chưa

Thao tác trên màn hình

1. Mở PowerShell.
2. Chạy:

```
node --version  
npm --version
```

Lời thoại

Trước khi cài, chúng ta kiểm tra máy đã có Node.js hay chưa. Nếu cả hai lệnh trả về số phiên bản, bạn có thể giữ bản đang dùng nếu project chạy ổn. Nếu PowerShell báo không nhận diện được lệnh `node`, hãy thực hiện bước cài đặt tiếp theo.

Kết quả mong đợi

```
v24.x.x  
11.x.x
```

Số patch có thể khác. Tại thời điểm biên soạn, Node.js 24 là nhánh LTS; trên trang tải hãy luôn chọn ô có nhãn **LTS**, không chọn **Current**, trừ khi giảng viên yêu cầu phiên bản khác. Repo cũng đã được thử nghiệm với Node.js 26, trong khi CI hiện dùng Node.js 20.

Bước 2.2 — Cài Node.js trên Windows

Thao tác trên màn hình

1. Mở <https://nodejs.org/en/download>.
2. Chọn **LTS** và đúng kiến trúc máy, thông thường là **Windows x64 Installer (.msi)**.
3. Mở file `.msi` vừa tải.
4. Chọn lần lượt **Next** → chấp nhận license → giữ thư mục mặc định.
5. Giữ các thành phần mặc định, đặc biệt là **npm package manager** và **Add to PATH**.
6. Chọn **Install** → **Finish**.
7. Đóng PowerShell cũ, mở một cửa sổ PowerShell mới và chạy lại:

```
node --version  
npm --version
```

Lời thoại

Chúng ta chọn bản LTS vì đây là nhánh ổn định dành cho đa số người dùng. Trình cài đặt Node.js đã đi kèm npm. Sau khi cài, cần mở terminal mới để hệ điều hành nạp lại biến PATH.

Bước 2.3 — Lệnh thay thế cho macOS/Linux

Callout trên màn hình, không cần quay toàn bộ

macOS với Homebrew:

```
brew install node@24
node --version
npm --version
```

macOS/Linux với `nvm` đã được cài:

```
nvm install --lts
nvm use --lts
node --version
npm --version
```

Với Linux, ưu tiên version manager như `nvm` thay vì cài một phiên bản Node.js cũ từ repository mặc định của distribution.

Điểm dừng xác nhận

- `node --version` trả về số phiên bản.
- `npm --version` trả về số phiên bản.
- Không còn lỗi “command not found” hoặc “is not recognized”.

Cảnh 3 — Cài Postman Desktop (03:30–05:40)

Bước 3.1 — Tải và cài Postman

Thao tác trên màn hình — Windows

1. Mở <https://www.postman.com/downloads/>.
2. Chọn bản **Windows 64-bit** hoặc **Windows ARM64** đúng với máy.
3. Chạy file cài đặt vừa tải.
4. Chờ Postman tự cài và khởi động.
5. Chọn đăng nhập/đăng ký tài khoản miễn phí nếu cần đồng bộ workspace; nếu chỉ gửi request cục bộ, có thể dùng lightweight API Client mà chưa đăng nhập.

Lời thoại

Nhóm sử dụng Postman Desktop vì ứng dụng desktop gửi request tới `localhost` trực tiếp và có đầy đủ Collection Runner. Tài khoản Postman hữu ích khi cần đồng bộ collection, nhưng không bắt buộc cho request cục bộ cơ bản.

Callout macOS/Linux

```
# macOS
brew install --cask postman

# Linux có Snap
sudo snap install postman
```

Không khởi chạy Postman bằng `sudo` trên Linux vì có thể tạo file cấu hình sai quyền sở hữu.

Bước 3.2 — Kiểm tra Postman

Thao tác trên màn hình

1. Chờ giao diện Postman mở hoàn tất.
2. Chọn **New** hoặc nút dấu cộng để tạo một HTTP Request trống.
3. Không gửi request ở video này; đóng tab request sau khi xác nhận giao diện hoạt động.

Lời thoại

Chỉ cần Postman mở được và tạo được tab HTTP Request là bước cài đặt đã thành công. Collection của repo sẽ được import trong video thực hành Postman.

Ghi chú

- Nếu dùng Postman Web thay vì app desktop, cần cài **Postman Desktop Agent** để tránh giới hạn của trình duyệt khi gọi API local.
- Repo có sẵn collection tại `src/postman/collections/` và environment tại `src/postman/environments/`.

Cảnh 4 — Cài VS Code và REST Client extension (05:40–07:40)

Bước 4.1 — Cài hoặc mở VS Code

Thao tác trên màn hình

1. Nếu chưa có VS Code, mở <https://code.visualstudio.com/download> và tải đúng bản cho hệ điều hành.
2. Trên Windows, khi cài nên bật **Add to PATH** và **Open with Code**.
3. Mở VS Code.

Lời thoại

VS Code được dùng để xem source code và gửi request trực tiếp từ file `.http`. Nếu đã cài sẵn VS Code, bạn có thể bỏ qua phần tải ứng dụng.

Bước 4.2 — Cài đúng extension REST Client

Thao tác trên màn hình

1. Mở tab **Extensions** bằng `Ctrl+Shift+X` trên Windows/Linux hoặc `Cmd+Shift+X` trên macOS.

2. Tìm chính xác: `REST Client`.
3. Chọn extension có:
4. Tên: **REST Client**
5. Publisher: **Huachao Mao**
6. Extension ID: `humao.rest-client`
7. Bấm **Install**.
8. Chờ nút đổi thành **Disable/Uninstall**, nghĩa là extension đã được cài.

Lời thoại

Marketplace có nhiều extension tên gần giống nhau. Hãy kiểm tra publisher Huachao Mao và ID `humao.rest-client` để cài đúng công cụ mà repo sử dụng.

Cách cài bằng terminal, dùng khi cần

```
code --install-extension humao.rest-client
```

Kết quả mong đợi

- VS Code hiển thị REST Client ở trạng thái **Installed**.
- Sau khi mở file `.http`, phía trên mỗi request sẽ có liên kết **Send Request**.

Cảnh 5 — Kiểm tra Git và clone repository (07:40–10:10)

Bước 5.1 — Kiểm tra hoặc cài Git

Thao tác trên màn hình

Mở terminal mới và chạy:

```
git --version
```

Nếu chưa có Git trên Windows, chọn một trong hai cách:

```
winget install --id Git.Git -e --source winget
```

Hoặc tải Git for Windows tại <https://git-scm.com/install/windows>. Sau khi cài, đóng và mở lại terminal.

Callout macOS/Linux

```
# macOS – thường kích hoạt Command Line Tools
xcode-select --install

# Ubuntu/Debian
sudo apt update
sudo apt install git
```

Lời thoại

Git là công cụ tải và quản lý source code. Khi lệnh `git --version` trả về số phiên bản, chúng ta đã sẵn sàng clone repo.

Bước 5.2 — Chọn thư mục và clone repo

Thao tác trên màn hình

1. Chuyển đến thư mục muốn lưu project, ví dụ thư mục `Documents`.
2. Chạy đúng URL HTTPS của repo:

```
cd $HOME\Documents
git clone https://github.com/se-bros/Software_Testing_api_contract_testing.git
cd Software_Testing_api_contract_testing
git remote -v
code .
```

macOS/Linux:

```
cd "$HOME/Documents"
git clone https://github.com/se-bros/Software_Testing_api_contract_testing.git
cd Software_Testing_api_contract_testing
git remote -v
code .
```

Lời thoại

Lệnh `git clone` tạo một thư mục mới tên `Software_Testing_api_contract_testing`. Sau đó mình đi vào thư mục này, kiểm tra remote `origin`, rồi mở toàn bộ repo bằng VS Code.

Kết quả mong đợi

```
origin https://github.com/se-bros/Software_Testing_api_contract_testing.git (fetch)
origin https://github.com/se-bros/Software_Testing_api_contract_testing.git (push)
```

Bước 5.3 — Xác nhận cấu trúc repo và REST Client

Thao tác trên màn hình

1. Trong Explorer của VS Code, mở `src/rest-client/product-service.http`.
2. Trỏ chuột vào liên kết **Send Request** phía trên request đầu tiên.
3. Không bấm gửi vì Provider API chưa được khởi động trong video này.
4. Mở nhanh các thư mục:
5. `src/sample-api/pact-workshop-js/`
6. `src/postman/`
7. `src/pact/`

Lời thoại

Khi thấy nút Send Request trong file `product-service.http`, chúng ta biết REST Client đã nhận đúng định dạng. Repo cũng có sẵn source API, collection Postman và phần contract testing bằng Pact.

Cảnh 6 — Đăng ký và thiết lập PactFlow Broker (10:10–13:50)

Giao diện SmartBear/PactFlow có thể thay đổi tên hoặc vị trí nút theo thời điểm. Nếu nhấn trên màn hình khác đôi chút, hãy tìm module **Contract Testing**, phần **Settings** và mục **API Tokens**.

Bước 6.1 — Tạo hoặc đăng nhập tài khoản

Thao tác trên màn hình

1. Mở <https://pactflow.io/>.
2. Chọn **Start free, Try for free** hoặc nút đăng ký tương đương.
3. Đăng ký bằng email hoặc tài khoản được trường/nhóm cung cấp.
4. Xác nhận email nếu hệ thống yêu cầu.
5. Đăng nhập vào SmartBear/Swagger Catalog.

Lời thoại

PactFlow là Pact Broker dạng dịch vụ. Sau khi đăng ký hoặc đăng nhập, tài khoản có thể được chuyển vào giao diện SmartBear hoặc Swagger Catalog. Đây là hành vi bình thường của nền tảng hiện tại.

Bước 6.2 — Tạo workspace Contract Testing

Thao tác trên màn hình

1. Trong sidebar hoặc catalog, chọn **Contract Testing**.
2. Chọn organization phù hợp. Nếu tự học, dùng organization cá nhân hoặc organization thử nghiệm.
3. Bấm **Set up workspace** nếu workspace chưa được khởi tạo.
4. Đặt tên workspace/organization dễ nhận biết, không chứa secret.
5. Chờ dashboard Contract Testing hiển thị.
6. Sao chép Broker URL có dạng:

```
https://<your-workspace>.pactflow.io
```

Lời thoại

Broker URL là địa chỉ duy nhất của workspace. Hãy sao chép toàn bộ URL có giao thức HTTPS và không thêm dấu gạch chéo ở cuối khi điền vào file cấu hình.

Bước 6.3 — Lấy API token đúng quyền

Thao tác trên màn hình

1. Vào **Settings** → **API Tokens**.
2. Xác định hai loại token nếu giao diện cung cấp:
3. **Read-only token**: chỉ phù hợp để đọc hoặc verify contract.
4. **Read/write token**: cần cho publish Pact và ghi kết quả verification.
5. Với repo này, chọn/copy **Read/write token** để chạy đầy đủ publish và verify.
6. Ngay trước khi hiện token, chèn overlay “**TOKEN ĐÃ ĐƯỢC CHE**” và blur vùng giá trị.
7. Lưu token tạm thời trong password manager hoặc clipboard; không lưu vào tài liệu, chat hay source code.

Lời thoại

Repo cần quyền ghi để publish contract và publish kết quả verification, vì vậy chúng ta dùng read/write token. Token có quyền truy cập workspace, cần được bảo vệ giống như mật khẩu. Trong video, toàn bộ giá trị token thật phải được che.

Chú thích dựng phim bắt buộc

- Zoom vào tên trường **Base URL**, **Read only token**, **Read/write token**.
- Không zoom hoặc giữ hình tại phần giá trị token.
- Nếu đã lộ token, regenerate ngay sau khi quay và không sử dụng token đó nữa.

Cảnh 7 — Cấu hình PactFlow trong repo (13:50–16:20)

Bước 7.1 — Tạo file `src/.env`

Thao tác trên màn hình — PowerShell

Tại thư mục gốc repo:

```
Copy-Item src/.env.example src/.env
code src/.env
```

macOS/Linux:

```
cp src/.env.example src/.env
code src/.env
```

Nội dung cần điền:

```
PACT_BROKER_BASE_URL=https://<your-workspace>.pactflow.io
PACT_BROKER_TOKEN=<your-read-write-token>
```

Lời thoại

Repo đã cung cấp file mẫu `src/.env.example`. Chúng ta sao chép thành `src/.env`, thay Broker URL và token bằng giá trị của workspace. Không thêm dấu nhảy nếu token không yêu cầu, và không commit file `.env`.

Bước 7.2 — Xác nhận `.env` không bị Git theo dõi

Thao tác trên màn hình

1. Trước khi chạy lệnh, thay token thật bằng placeholder hoặc che vùng editor.
2. Chạy:

```
git check-ignore -v src/.env
git status --short
```

Kết quả mong đợi

- `git check-ignore` chỉ ra rule `.env` trong `.gitignore`.
- `git status --short` không liệt kê `src/.env`.

Lời thoại

Bước kiểm tra này bảo đảm Git đang bỏ qua file secret. Nếu `src/.env` xuất hiện trong `git status`, dừng lại và không commit cho đến khi kiểm tra lại `.gitignore`.

Bước 7.3 — Nạp biến môi trường và kiểm tra Broker

Phương án quay an toàn

- Dừng ghi màn hình khi thay placeholder bằng token thật.
- Nạp biến trong terminal mới.
- Bật ghi màn hình trở lại sau khi terminal đã sẵn sàng.
- Chỉ chạy lệnh kiểm tra HTTP; tuyệt đối không in biến token.

PowerShell:

```
Get-Content src/.env | Where-Object { $_ -match '^[^#].+= ' } | ForEach-Object {
    $name, $value = $_ -split '=', 2
    Set-Item -Path "Env:$name" -Value $value
}

$headers = @{ Authorization = "Bearer $env:PACT_BROKER_TOKEN" }
(Invoke-WebRequest -Uri $env:PACT_BROKER_BASE_URL -Headers $headers).StatusCode
```

macOS/Linux (`bash` hoặc `zsh`):

```
set -a
source src/.env
set +a
curl --fail --silent --output /dev/null --write-out '%{http_code}\n' \
  --header "Authorization: Bearer $PACT_BROKER_TOKEN" \
  "$PACT_BROKER_BASE_URL"
```

Kết quả mong đợi

```
200
```

Lời thoại

Mã 200 cho biết Broker URL và token đang hợp lệ. Lệnh chỉ in HTTP status, không in token hay nội dung nhảy cảm. Các script Pact của repo đọc hai biến môi trường này khi publish hoặc verify contract.

Lưu ý: file `.env` không được các script Node.js trong repo tự động nạp. Trước khi chạy lệnh Pact ở terminal mới, cần nạp biến như bước trên hoặc cấu hình chúng trong môi trường CI.

Bước 7.4 — Giới thiệu cấu hình GitHub Actions, không nhập secret trong video

Thao tác trên màn hình

1. Mở `.github/workflows/pact-verification.yml` trong VS Code.
2. Highlight hai biến:

```
PACT_BROKER_BASE_URL: ${ secrets.PACT_BROKER_BASE_URL }
PACT_BROKER_TOKEN: ${ secrets.PACT_BROKER_TOKEN }
```

Lời thoại

Khi chạy CI, hai giá trị này phải được tạo dưới dạng GitHub Actions Secrets, không ghi trực tiếp vào workflow. Video CI/CD sẽ hướng dẫn phần cấu hình trên GitHub.

Cảnh 8 — Tổng kết (16:20–17:00)

Hình ảnh trên màn hình

- Hiện checklist sáu mục, lần lượt đánh dấu hoàn thành.
- Kết thúc ở VS Code với repo đang mở; không để file `.env` hiển thị.

Lời thoại

Chúng ta đã hoàn tất môi trường: Node.js và npm hoạt động, Postman mở được, REST Client đã cài đúng, repository đã được clone và PactFlow Broker trả kết nối thành công. Ở video tiếp theo, chúng ta sẽ khởi động Product Service và bắt đầu gửi request kiểm thử. Trước khi kết thúc, hãy chắc chắn file `src/.env` không nằm trong Git và token không xuất hiện trong bản ghi màn hình.

5. Bảng kiểm kết thúc video

Người quay và người review cần xác nhận toàn bộ mục sau:

- [] `node --version` chạy thành công.
- [] `npm --version` chạy thành công.
- [] Postman Desktop mở và tạo được HTTP Request mới.
- [] VS Code mở được repository.
- [] REST Client có ID `humao.rest-client` và ở trạng thái Installed.
- [] File `src/rest-client/product-service.http` hiển thị **Send Request**.
- [] `git remote -v` trở đến repo `se-bros/Software_Testing_api_contract_testing`.
- [] PactFlow Contract Testing workspace đã được tạo.
- [] Broker URL có dạng `https://<workspace>.pactflow.io`.
- [] Đã dùng read/write token cho luồng publish/verify của repo.
- [] Broker trả HTTP `200` khi kiểm tra bằng bearer token.
- [] `git check-ignore -v src/.env` xác nhận file bị ignore.
- [] Token thật không xuất hiện trong video, phụ đề, clipboard popup hoặc terminal history.

6. Xử lý lỗi thường gặp

Lỗi `node`, `npm`, `git` hoặc `code` không được nhận diện

Nguyên nhân thường gặp: terminal được mở trước khi cài đặt hoặc công cụ chưa được thêm vào `PATH`.

Cách xử lý:

1. Đóng toàn bộ PowerShell/Terminal và VS Code.
2. Mở terminal mới rồi thử lại.
3. Trên Windows, restart máy nếu PATH vẫn chưa được cập nhật.
4. Nếu chỉ lệnh `code` lỗi, trong VS Code mở Command Palette và chạy **Shell Command: Install 'code' command in PATH** trên macOS; trên Windows sửa/cài lại VS Code với tùy chọn **Add to PATH**.

Clone repo báo `destination path ... already exists`

Nguyên nhân: thư mục đích đã tồn tại.

Cách xử lý an toàn: không xóa thư mục ngay. Mở thư mục đó, chạy `git remote -v` và kiểm tra có đúng repo hay không. Nếu muốn clone mới, chọn một thư mục cha khác hoặc đặt tên đích mới:

```
git clone https://github.com/se-bros/Software_Testing_api_contract_testing.git seminar-api-testing
```

File `.http` không có nút **Send Request**

1. Kiểm tra REST Client đã Enabled trong workspace hiện tại.
2. Kiểm tra đúng extension ID `humao.rest-client`.
3. Reload VS Code bằng lệnh **Developer: Reload Window**.

4. Xác nhận file có đuôi `.http` hoặc `.rest`.

Postman Web không gọi được `localhost`

Dùng Postman Desktop hoặc cài và chọn Postman Desktop Agent. Trình duyệt có thể bị giới hạn bởi CORS hoặc không có agent local phù hợp.

PactFlow trả `401 Unauthorized` hoặc `403 Forbidden`

1. Kiểm tra URL là Broker URL của đúng workspace, không phải URL trang marketing.
2. Kiểm tra token chưa hết hạn hoặc bị regenerate.
3. Dùng read/write token nếu thao tác cần publish.
4. Không thêm khoảng trắng, dấu nháy hoặc ký tự xuống dòng vào token.
5. Tạo token mới nếu token cũ đã từng xuất hiện công khai.

PactFlow không trả `200`

- `401/403`: thông tin xác thực hoặc quyền token không hợp lệ.
- `404`: thường dùng sai Broker URL.
- Lỗi DNS/TLS: kiểm tra mạng, VPN, proxy hoặc firewall của trường/công ty.
- `Invoke-WebRequest` báo lỗi nhưng trình duyệt mở được: kiểm tra proxy của PowerShell hoặc thử `curl.exe` trên Windows.

`src/.env` xuất hiện trong `git status`

Không commit. Xác nhận `.gitignore` của repo có các rule:

```
.env
.env.*
!.env.example
```

Nếu file đã từng bị add nhầm vào Git index, cần nhờ người phụ trách repo xử lý và rotate token trước khi tiếp tục.

7. Tài liệu tham khảo

Các nguồn sau được đối chiếu khi biên soạn ngày 28/07/2026:

- [Node.js — Download](#)
- [Postman — Install Postman](#)
- [Visual Studio Marketplace — REST Client](#)
- [Git — Downloads](#)
- [Pact Broker CLI — Broker URL và bearer token](#)
- [PactFlow](#)
- Repo-local: `src/.env.example`, `src/pact/README.md`, `.github/workflows/pact-verification.yml`

Thông tin Node.js, Postman và PactFlow đã được kiểm tra qua Context7 và đối chiếu với tài liệu chính thức. Nhấn nút trên giao diện web có thể thay đổi sau ngày cập nhật tài liệu.